ADVANCED PYTHON PROGRAMMING

LAB ASSIGNMENT – 4

Name: G K Monika

Reg.No: 23MID0002

The following file consists of these assignments:

1. Leetcode questions using classes [6 questions]
2. Multithreading tasks [7 Tasks]
3. MultiThreading using priority queue
4. Printing main thread name and worker threads

[CSI3007] ADVANCED PYTHON PROGRAMMING

ASSIGNMENT – OOPS rephrasing of LeetCode Questions

NAME: G K MONIKA

REG.NO: 23MID0002

EASY LEVEL QUESTIONS

LEETCODE QUESTION 67: https://leetcode.com/problems/add-binary/

ADD BINARY

Question:
Create a class BinaryCalculator that represents a calculator for binary numbers. The class should accept two binary strings and provide a method to add them without converting them into decimal numbers. Return the result as a binary string.

Sample Test Cases:

Example 1:

```
Input: a = "11", b = "1"
Output: "100"
```

Example 2:

```
Input: a = "1010", b = "1011"
Output: "10101"
```

LOGIC

Traverse both binary strings from right to left, add corresponding bits along with the carry, and construct the result from right to left.

Benefit of OOP in this Question

The BinaryCalculator class keeps the binary numbers and addition operation together, making the calculation reusable.

LEETCODE QUESTION 69: https://leetcode.com/problems/sqrtx/

SQRT(X)

Question:
Create a class SquareRootCalculator that represents a mathematical calculator. The class should accept a non-negative integer and provide a method to calculate its integer square root without using built-in square-root or power functions.

Sample Test Cases

Example 1:

```
Input: x = 4

Output: 2

Explanation: The square root of 4 is 2, so we return 2.
```

Example 2:

```
Input: x = 8

Output: 2

Explanation: The square root of 8 is 2.82842..., and since we round it down
to the nearest integer, 2 is returned.
```

LOGIC

Use binary search to find the largest integer whose square is less than or equal to the given number.

Benefit of OOP in this Question

The class encapsulates the number and square-root calculation, keeping the mathematical operation organized and reusable.

MEDIUM LEVEL QUESTIONS

LEETCODE QUESTION 192: https://leetcode.com/problems/word-frequency/

WORD FREQUENCY

Question:

Create a class WordFrequencyAnalyzer that represents a text-analysis tool. The class should accept a text containing multiple words and provide a method to count how many times each word occurs. Display the words along with their frequencies in descending order of frequency.

Sample Test Cases

Example:

Assume that words.txt has the following content:

```
the day is sunny the the
the sunny is is
```

Your script should output the following, sorted by descending frequency:

```
the 4
is 3
sunny 2
day 1
```

LOGIC

Read each word, maintain its count using a frequency map, and sort the words based on their frequency in descending order.

Benefit of OOP in this Question

The class can manage the text, frequency data, counting process, and result generation in one organized unit.

LEETCODE QUESTION 287: https://leetcode.com/problems/find-the-duplicate-number/

FIND THE DUPLICATE NUMBER

Question:

Create a class DuplicateDetector that represents a tool for finding repeated numbers. The class should accept an integer array containing n + 1 integers, where every integer is between 1 and n. Exactly one number is repeated. Provide a method to find and return the duplicate number.

Sample Test Cases

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

LOGIC

Use an efficient technique such as Floyd's cycle detection algorithm to find the duplicate without modifying the array.

Benefit of OOP in this Question

The DuplicateDetector class keeps the array and duplicate-finding algorithm together, making the detector reusable for different arrays.

HARD LEVEL QUESTIONS

LEETCODE QUESTION 214: https://leetcode.com/problems/shortest-palindrome/

SHORTEST PAINDROME

Question:
Create a class PalindromeBuilder that represents a string transformation tool. The class should accept a string and provide a method that adds the minimum number of characters **only to the beginning** of the string to make it a palindrome.

Sample Test Cases

Example 1:

```
Input: s = "aacecaaa"
Output: "aaacecaaa"
```

Example 2:

```
Input: s = "abcd"
Output: "dcbabcd"
```

LOGIC

Find the longest palindromic prefix of the string. Reverse the remaining suffix and add it to the beginning to obtain the shortest palindrome.

Benefit of OOP in this Question

The class can store the string and handle palindrome checking and construction as related operations.

LEETCODE QUESTION 1363: https://leetcode.com/problems/largest-multiple-of-three/

LARGEST MULTIPLE OF THREE

Question:

Create a class MultipleOfThreeGenerator that manages a collection of digits. The class should accept an array of digits and provide a method to select and rearrange some or all of them to form the largest possible number divisible by 3.

Return the resulting number as a string.

Sample Test Cases

Example 1:

```
Input: digits = [8,1,9]
Output: "981"
```

Example 2:

```
Input: digits = [8,6,7,1,0]
Output: "8760"
```

Example 3:

```
Input: digits = [1]
Output: ""
```

LOGIC

Calculate the sum of all digits and remove the minimum necessary digits so that the remaining digit sum is divisible by 3. Sort the remaining digits in descending order to form the largest number.

Benefit of OOP in this Question

The class encapsulates the digit collection and the logic for selecting, arranging, and generating the final number.

In [1]:
```
import threading #imports thread library
import time #imports time library
```

In [25]:
```
#Task 1-23MID0002
#Create and execute a single thread
start=time.perf_counter()
def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    print('Done sleeping...')

t1=threading.Thread(target=do_something,args=[1])
t1.start()
t1.join()
end=time.perf_counter()
```

```
sleeping 1 second (s)
Done sleeping...
```

In [7]:
```
#Task 2-23MID0002
#Create and execute a multiple threads
start=time.perf_counter()
for _ in range(10):
    t=threading.Thread(target=do_something,args=[2])
    t.start()
    ths.append(t)
for i in ths:
    i.join()
end=time.perf_counter()
print(f'Finished in {end-start} secs')
```

```
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.002277089999552 secs
```

In [26]:
```
#Task 3-23MID0002
#use concurrent futures to execute a single thread
import concurrent.futures #imports concurrent futures library

start=time.perf_counter() #captures the time
with concurrent.futures.ThreadPoolExecutor() as exec:

    t=exec.submit(do_something,2)
    t.result()
end=time.perf_counter()
print(f'Finished in {end-start} secs')
```

```
sleeping 2 second (s)
Done sleeping...
Finished in 2.000990387998172 secs
```

In [27]:
```
#Task 4-23MID0002
#use concurrent futures to execute multiple threads
ths=[]
```

```
start=time.perf_counter() #captures the time
with concurrent.futures.ThreadPoolExecutor() as exec:
    for _ in range(10):
        th=exec.submit(do_something,2)
        ths.append(th)
    for i in ths:
        i.result()
    end=time.perf_counter()
    print(f'Finished in {end-start} secs')
```

```
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.0020515129981504 secs
```

In [28]:
```
#Task 5-23MID0002
#Using list comprehension for multithreading
ths=[]
start=time.perf_counter() #captures the time
with concurrent.futures.ThreadPoolExecutor() as exec:
    ths=[exec.submit(do_something,2) for _ in range(10)]
    for i in ths:
        i.result()
```

```
    end=time.perf_counter()
    print(f'Finished in {end-start} secs')
```

```
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.0022754530000384 secs
```

In [30]:
```
#Task 6-23MID0002
#Using map function for multithreading
ths=[]
res=[]
slist=[1,2,3,1,2,3,1,2,3,1]
start=time.perf_counter() #captures the time
with concurrent.futures.ThreadPoolExecutor() as exec:
    ths=exec.map(do_something,slist)
    res=[i.result() for _ in ths]
    print(res)
end=time.perf_counter()
print(f'Finished in {end-start} secs')
```

```
sleeping 1 second (s)
sleeping 2 second (s)
sleeping 3 second (s)
sleeping 1 second (s)
sleeping 2 second (s)
sleeping 3 second (s)
sleeping 1 second (s)
sleeping 2 second (s)
sleeping 3 second (s)
sleeping 1 second (s)
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...Done sleeping...
Done sleeping...

Done sleeping...
Done sleeping...
Done sleeping...
[None, None, None, None, None, None, None, None, None, None]
Finished in 3.0026701310016506 secs
```

In [31]:
```python
#Task 7
import requests
vi_urls=[
    'https://pixabay.com/videos/netherlands-dwingeloo-autumn-nature-370702',
    'https://pixabay.com/videos/solar-eclipse-sun-moon-370344',
    'https://pixabay.com/videos/golden-horn-beach-spit-sand-sandy-270941',
    'https://pixabay.com/videos/cherry-blossom-ornamental-cherry-340838',
    'https://pixabay.com/videos/dandelion-flower-blossom-bloom-348656',
    'https://pixabay.com/videos/water-dark-background-sea-deep-304019',
    'https://pixabay.com/videos/butterfly-nature-insect-fauna-345137',
    'https://pixabay.com/videos/starry-sky-milky-way-cloud-285663',
    'https://pixabay.com/videos/winter-snow-nature-cold-frost-336603',
    'https://pixabay.com/videos/penguin-nature-animal-wildlife-327101'
]

t1=time.perf_counter()
def download_urls(vi_urls):
    vid_bytes=requests.get(vi_urls).content
```

```
    vid_name=vi_urls.split('/')[4]
    vid_name=f'{vid_name}.mp4'
    with open(vid_name,'wb') as vid_file:
        vid_file.write(vid_bytes)
        print(f'{vid_name} was download...')

with concurrent.futures.ThreadPoolExecutor() as exec:
    exec.map(download_urls,vi_urls)

t2=time.perf_counter()
print(f'Finished in {t2-t1} seconds')
```

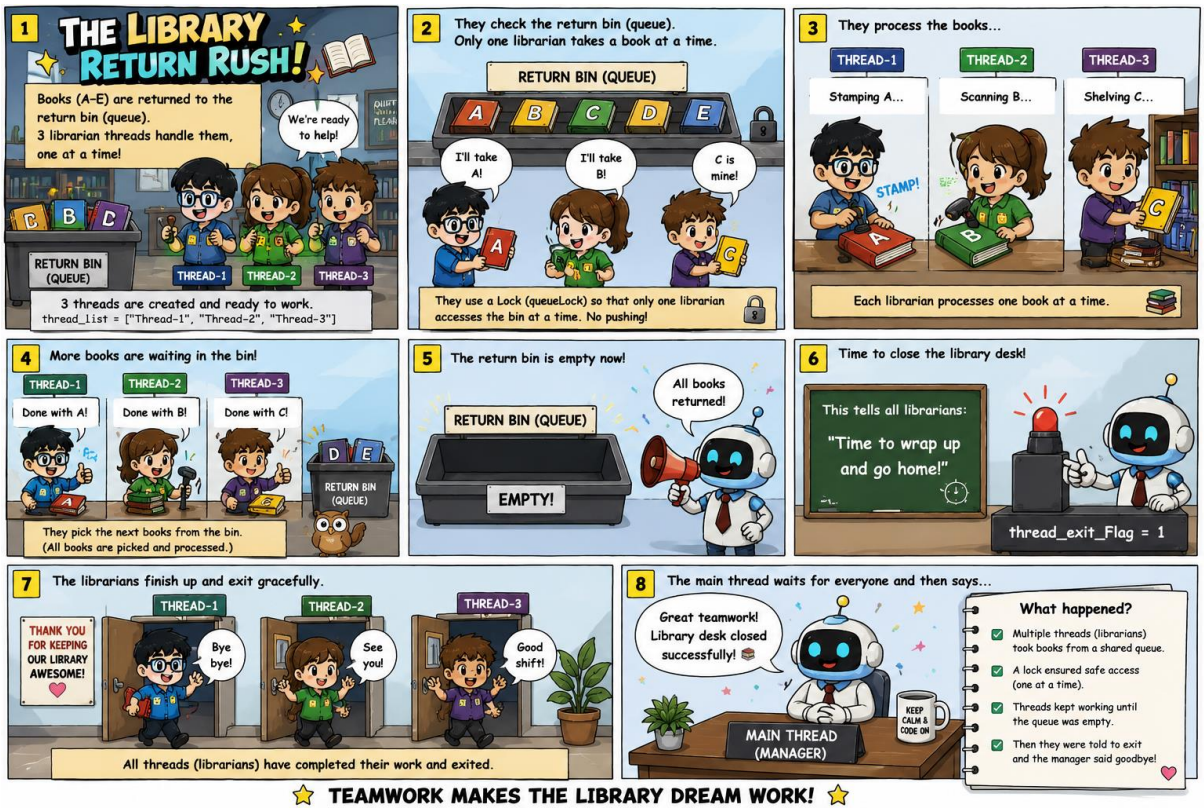
```
golden-horn-beach-spit-sand-sandy-270941.mp4 was download...butterfly-nature-insect-fauna-345137.mp4 was download...

netherlands-dwingeloo-autumn-nature-370702.mp4 was download...
dandelion-flower-blossom-bloom-348656.mp4 was download...
penguin-nature-animal-wildlife-327101.mp4 was download...
starry-sky-milky-way-cloud-285663.mp4 was download...
solar-eclipse-sun-moon-370344.mp4 was download...
winter-snow-nature-cold-frost-336603.mp4 was download...
water-dark-background-sea-deep-304019.mp4 was download...
cherry-blossom-ornamental-cherry-340838.mp4 was download...
Finished in 0.1854637479991652 seconds
```

In [ ]:

MULTITHREADING WITH PRIORITY QUEUE

STORYBOARD:



CODE AND OUTPUTS ARE IN THE FOLLOWING JUPYTER FILE

In [7]:
```
print("G.K.Monika-23MID0002--Multithreading priority Queue")
```

```
G.K.Monika-23MID0002--Multithreading priority Queue
```

In [1]:
```
import queue
import threading
import time
```

In [2]:
```
thread_exit_Flag = 0
class sample_Thread (threading.Thread):
    def __init__(self, threadID, name, q):
        threading.Thread.__init__(self)
        self.threadID = threadID
        self.name = name
        self.q = q
    def run(self):
        print ("initializing " + self.name)
        process_data(self.name, self.q)
        print ("Exiting " + self.name)
```

In [3]:
```
# helper function to process data
def process_data(threadName, q):
    while not thread_exit_Flag:
        queueLock.acquire()
        if not workQueue.empty():
            data = q.get()
            queueLock.release()
            print ("% s processing % s" % (threadName, data))
        else:
            queueLock.release()
            time.sleep(1)
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1
```

In [4]:
```
# Create new threads
for thread_name in thread_list:
    thread = sample_Thread(threadID, thread_name, workQueue)
    thread.start()
    threads.append(thread)
    threadID += 1
```

```
initializing Thread-1
initializing Thread-2
initializing Thread-3
```

In [5]:
```
# Fill the queue
queueLock.acquire()
for items in name_list:
    workQueue.put(items)
queueLock.release()
```

```
Thread-1 processing A
Thread-1 processing B
Thread-1 processing C
Thread-1 processing D
Thread-1 processing E
```

In [6]:
```
# Wait for the queue to empty
while not workQueue.empty():
    pass
# Notify threads it's time to exit
thread_exit_Flag = 1
# Wait for all threads to complete
for t in threads:
    t.join()
print ("Exit Main Thread")
```

```
Exiting Thread-1
Exiting Thread-2
Exiting Thread-3
Exit Main Thread
```

In [ ]:

In [11]:
```
print("G K MONIKA [23MID0002] - PRINTING MAIN THREAD NAME AND WORKER THREADS")
```

```
G K MONIKA [23MID0002] - PRINTING MAIN THREAD NAME AND WORKER THREADS
```

In [18]:
```
import queue
import threading
import time

# 1. Print the Main Thread name right at the start
main_thread_name = threading.current_thread().name
print(f"--- Starting Application on: {main_thread_name} ---\n")

# Use a dictionary or object to share the flag cleanly across threads
state = {"thread_exit_Flag": 0}

class SampleThread(threading.Thread):
    def __init__(self, threadID, name, q):
        super().__init__()
        self.threadID = threadID
        self.name = name  # Assign custom name to the worker thread
        self.q = q

    def run(self):
        # 2. Print worker thread name upon initialization
        print(f"[{self.name}] Initializing...")
        process_data(self.name, self.q)
        print(f"[{self.name}] Exiting.")
```

```
--- Starting Application on: MainThread ---
```

In [19]:
```
def process_data(threadName, q):
    while not state["thread_exit_Flag"]:
        queueLock.acquire()
        if not workQueue.empty():
            data = q.get()
            queueLock.release()
            # 3. Print worker thread name alongside the data it processes
            print(f"[{threadName}] is processing data item: {data}")
            time.sleep(0.1) # Brief pause to allow output to stagger cleanly
        else:
            queueLock.release()
            time.sleep(0.2)
```

In [20]:
```
# Configurations
thread_list = ["WorkerThread-1", "WorkerThread-2", "WorkerThread-3"]
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1
```

In [21]:
```
# Create and start worker threads
for thread_name in thread_list:
    thread = SampleThread(threadID, thread_name, workQueue)
    thread.start()
    threads.append(thread)
    threadID += 1
```

```
[WorkerThread-1] Initializing...
[WorkerThread-2] Initializing...
[WorkerThread-3] Initializing...
```

In [22]:
```python
# Safely load data into the queue
queueLock.acquire()
for items in name_list:
    workQueue.put(items)
queueLock.release()

# Wait for worker threads to consume queue items
while not workQueue.empty():
    pass

# Signal threads to wrap up execution
state["thread_exit_Flag"] = 1

# Merge threads back into the main program flow
for t in threads:
    t.join()

# 4. Final print confirming control is back on the Main Thread
print(f"\n--- Back on: {threading.current_thread().name} ---")
```

```
[WorkerThread-1] is processing data item: A
[WorkerThread-2] is processing data item: B
[WorkerThread-3] is processing data item: C
[WorkerThread-1] is processing data item: D
[WorkerThread-2] is processing data item: E
[WorkerThread-3] Exiting.
[WorkerThread-1] Exiting.
[WorkerThread-2] Exiting.

--- Back on: MainThread ---
```

In [ ]: